Computación y estructuras discretas I

Unidad 3: Estructuras Discretas no recursivas y análisis de algoritmos – Pilas y Colas

Juan Marcos Caicedo Mejía – jmcaicedo@icesi.edu.co

Departamento de Computación y Sistemas Inteligentes
Facultad Barberi de Ingeniería, Diseño y Ciencias Aplicadas
Universidad ICESI
Material adaptado del material original del profesor Oscar Bedoya

2026

Tabla de contenido

1. Pilas

2. Colas

Contenido

1. Pilas

2. Colas

Estructuras de datos

Pila

Una pila es una estructura de datos tipo LIFO (Last In First Out), por lo que el último elemento insertado será el primero en ser borrado.

Operaciones básicas:

```
STACK-EMPTY(S)
PUSH(S,x)
POP(S)
```

Estructuras de datos

Pila

Una forma de implementar la pila es por medio de un arreglo unidimensional

	1	2	3	4	5
S	10	4	5		

Esto supone varios aspectos:

- La pila tiene una capacidad limitada
- Se cuenta con un atributo adicional, llamado `top[S]`, que almacena el índice en el arreglo que guarda el último valor, esto es, el tope de la pila
- Cuando la pila esté vacía, `top[S]=0`

Estructuras de datos

	1	2	3	4	5
S	10	4	5		

top[S]=3

Estructuras de datos

	1	2	3	4	5
S	10	4	5		

top[S]=3

Indique lo que sucede después de cada instrucción, siendo S la pila que se muestre arriba:

```
STACK-EMPTY(S)
PUSH(S,4)
PUSH(S,12)
PUSH(S,7)
```

Estructuras de datos

	1	2	3	4	5
S	10	4	5		

top[S]=3

Indique lo que sucede después de cada instrucción, siendo S la pila que se muestre arriba:

```
STACK-EMPTY(S)
PUSH(S,4)
PUSH(S,12)
PUSH(S,7)
```

$\longrightarrow$ Overflow – desbordamiento en su capacidad máxima

Estructuras de datos

	1	2	3	4	5
S	10	4	5		

top[S]=3

Indique lo que sucede después de cada instrucción, siendo S la pila que se muestre arriba:

```
STACK-EMPTY(S)
POP(S)
POP(S)
POP(S)
STACK-EMPTY(S)
POP(S) ⟶ Underflow
```

Estructuras de datos

Indique un algoritmo para cada operación básica, acompañado de su respectiva complejidad usando la notación O

- `STACK-EMPTY(S)`, retorna true o false
- `PUSH(S,x)`, adiciona x al tope, no devuelve ningún valor
- `POP(S)`, borra el elemento que esté en el tope y devuelve ese valor

Estructuras de datos

```
STACK-EMPTY(S)
if (top[S]==0)
then return true
else return false
```

Estructuras de datos

```
STACK-EMPTY(S)
if (top[S]==0)
then return true
else return false
```

$T(n) = O(1)$, tiempo constante

Estructuras de datos

```
PUSH(S,x)
if (top[S] == n)
then error ``overflow''
else top[S] = top[S] + 1
     S[top[S]] = x
```

Estructuras de datos

```
PUSH(S,x)
if (top[S] == n)
then error ``overflow''
else top[S] = top[S] + 1
     S[top[S]] = x
```

$T(n) = O(1)$, tiempo constante

Estructuras de datos

```
POP(S)
if (top[S] == 0)
then error ``underflow''
else top[S] = top[S] - 1
     return S[top[S] + 1]
```

Estructuras de datos

```
POP(S)
if (top[S] == 0)
then error ``underflow''
else top[S] = top[S] - 1
      return S[top[S] + 1]
```

$T(n) = O(1)$, tiempo constante

Contenido

1. Pilas

2. Colas

Estructuras de datos

Cola

Una cola es una estructura de datos tipo FIFO (First In First Out), por lo que el primer elemento que es insertado, es el primero en ser borrado.

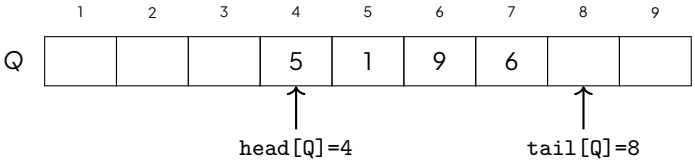
Operaciones básicas:

`ENQUEUE(Q,x)`

`DEQUEUE(Q)`

Cola

Una forma de implementar la cola es por medio de un arreglo unidimensional

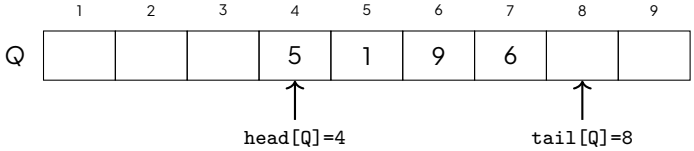


Esto supone varios aspectos:

- La cola tiene una capacidad limitada
- Se cuenta con dos atributos adicionales, `head[Q]` que guarda el índice de la cabeza y `tail[Q]` que apunta al siguiente lugar en el cual será insertado un elemento

Cola

Una forma de implementar la cola es por medio de un arreglo unidimensional




```
ENQUEUE(Q,2)
ENQUEUE(Q,8)
ENQUEUE(Q,3)
```

Cola

Una forma de implementar la cola es por medio de un arreglo unidimensional

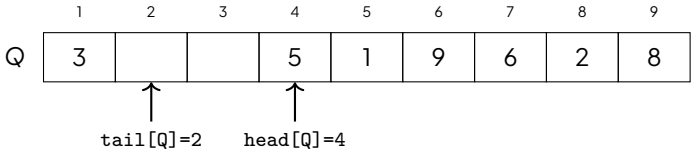
	1	2	3	4	5	6	7	8	9
Q	3			5	1	9	6	2	8
		↑		↑					
		`tail[Q]=2`		`head[Q]=4`					

```
ENQUEUE(Q,2)
ENQUEUE(Q,8)
ENQUEUE(Q,3)
```

⟶ Si se llega al final del arreglo, se intenta insertar en la posición 1

Cola

Una forma de implementar la cola es por medio de un arreglo unidimensional

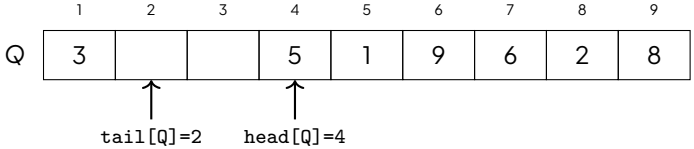


¿Cómo sabe que la cola está llena?

Estructuras de datos

Cola

Una forma de implementar la cola es por medio de un arreglo unidimensional



¿Cómo sabe que la cola está llena?

`tail[Q]=head[Q]`

Cola

Una forma de implementar la cola es por medio de un arreglo unidimensional

	1	2	3	4	5	6	7	8	9
Q	3			5	1	9	6	2	8
		↑		↑					
		`tail[Q]=2`		`head[Q]=4`					

¿Cómo sabe que la cola está llena?

`tail[Q]=head[Q]`

Inicialmente `tail[Q]=head[Q]=1`

Estructuras de datos

Indique un algoritmo para cada operación básica, acompañado de su respectiva complejidad usando la notación O

- `ENQUEUE(Q,x)`
- `DEQUEUE(Q)`

Estructuras de datos

```
ENQUEUE(Q,x)
if (tail[Q] == head[Q])
then error ``overflow''
else Q[tail[Q]] = x
     if (tail[Q] == n)
        then tail[Q] = 1
        else tail[Q] = tail[Q] + 1
```

Estructuras de datos

```
ENQUEUE(Q,x)
if (tail[Q] == head[Q])
then error ``overflow''
else Q[tail[Q]] = x
      if (tail[Q] == n)
         then tail[Q] = 1
         else tail[Q] = tail[Q] + 1
```

$T(n) = O(1)$, tiempo constante

Estructuras de datos

```
DEQUEUE(Q)
if (tail[Q] == head[Q])
then error ``underflow''
else x = Q[head[Q]]
      if (head[Q] == n)
         then head[Q] = 1
         else head[Q] = head[Q] + 1
      return x
```

Estructuras de datos

```
DEQUEUE(Q)
if (tail[Q] == head[Q])
then error ``underflow''
else x = Q[head[Q]]
      if (head[Q] == n)
         then head[Q] = 1
         else head[Q] = head[Q] + 1
      return x
```

$T(n) = O(1)$, tiempo constante